CPTS 360 Programming Assignment 3: Introduction to Linux Kernel Programming
Isaiah Foster, WSU ID: 11796849

1 Initialization and Cleanup

kmlab_init() creates the directory with *proc_mkdir()* and the entry */proc/kmlab/status* with *proc_create()* and mode 0666, then initializes the work item and starts the timer. *kmlab_exit()* stops each user of the list before freeing it. It removes the /proc entries so no callback can add a process, stops the timer with *del_timer_sync()* so no new work can be scheduled, cancels pending work with *cancel_work_sync()*, and frees every remaining list node.

2 Registered Process List and Locking

Each registered process is a *proc_list* node holding its PID and CPU time, stored in a kernel linked list (*LIST_HEAD*, *list_add_tail()*, *list_del()*). The /proc callbacks and the work function run concurrently, so every access to the list while the module is running holds a spinlock taken with *spin_lock_irqsave()*. Critical sections never sleep: *kmalloc()* and the user copies happen outside the lock.

3 /proc Write and Read Callbacks

kmlab_proc_write() registers a process. It copies the string into a kernel buffer with *copy_from_user()*, parses the PID with *kstrtouint()*, allocates a node, and adds it to the list under the lock. A string too long for the buffer or not a valid number returns *-EINVAL*.

kmlab_proc_read() allocates a one-page kernel buffer, formats one "PID: CPU time" line per process into it under the lock, then releases the lock and uses *copy_to_user()* to return the data starting at the file offset. It advances the offset by the amount copied, so a read at the end returns 0 and *cat* terminates. As the assignment allows, it does not support page breaks: output beyond one page is cut off.

4 Timer and Workqueue

Kernel timers are single shot, so *update_timer_callback()* re-arms itself with *mod_timer()* at *jiffies + 5 * HZ*. The callback runs in interrupt context and does only this and *schedule_work()*, which hands the update to a kernel worker thread. The timer runs even when no process is registered, which the assignment permits.

update_work_function() traverses the list under the lock with *list_for_each_entry_safe()*, which allows the current node to be deleted. It calls *get_cpu_use()* for each process. A return of 0 stores the new CPU time in the node, and any other return means the process has terminated, so its node is removed and freed.

5 User Application

userapp writes its PID to the status file, repeatedly computes a sum of squares until the time given on the command line has passed (10 seconds by default), then reads the status file and prints it to the screen.

6 Output

Both screenshots are from an Ubuntu 20.04 virtual machine running kernel 5.15.0-67. CPU times are the values of *task->utime*, in nanoseconds. In the one-process case, CPU time grows between the two reads, and *userapp* prints the list when it finishes. The list is empty after the process exits because the work function removed it.

```
-rw-rw-rw- 1 root root 0 Nov  5 20:11 /proc/kmlab/status
[1] 13535
== read 1 ==
13535: 5030507607
== read 2 ==
13535: 10145547833
== userapp output ==
13535: 10145547833
[1]+  Done                    ./userapp 12
== after exit ==
```

The two-process case runs the provided *kmlab_test.sh*, starting one process that runs for 5 seconds and one that runs for 35. The two lines before “read 2” are printed by the 5-second process when it finishes. Read 2 still lists that process because the next update had not yet run, and read 3 shows only the 35-second process. The second line under “read 3” is printed by that process when it finishes, and the list is empty after both exit. The final line is *ls /proc/kmlab* after *rmmod*, showing that removing the module removes the /proc directory.

```
== read 1 ==
13857: 0
13856: 0
13857: 5171631090
13856: 5163015740
== read 2 ==
13857: 5171631090
13856: 5163015740
== read 3 ==
13857: 20524871089
13857: 30763024198
== after both exit ==
ls: cannot access '/proc/kmlab': No such file or directory
```